

Form Validation with JavaScript Review

Validating Forms with JavaScript

- **Constraint Validation API:** Certain HTML elements, such as the `textarea` and `input` elements, expose a constraint validation API. This API allows you to assert that the user's provided value for that element passes any HTML-level validation you have written, such as minimum length or pattern matching.
- **`checkValidity()` method:** This method returns `true` if the element matches all HTML validation (based on its attributes), and `false` if it fails.

```
1 <form>
2   <label>
3     Email:
4     <input
5       id="email"
6       type="email"
7       required
8       pattern=".+\.com$"
9       placeholder="example@site.com"
10    />
11  </label>
12 </form>
13
14 <script>
15   const input = document.getElementById("email");
16
17   input.addEventListener("input", (e) => {
18     if (!e.target.checkValidity()) {
19       e.target.setCustomValidity("You must use a .com email.");
20     } else {
21       e.target.setCustomValidity("");
22     }
23   });
```

Email:

[Open Sandbox](#)

- **`reportValidity()` Method:** This method tells the browser that the `input` is invalid.

```
1 <form>
2   <label>
3     Email:
4     <input
5       id="email2"
6       type="email"
7       required
8       pattern=".+\.com$"
```



```
16
17   input.addEventListener("input", (e) => {
18     if (!e.target.checkValidity()) {
19       e.target.reportValidity();
20     }
21   });
```

Email:

[Open Sandbox](#)

- **validity Property:** This property is used to get the validity state of form controls (like `<input>`, `<select>`, etc.) and provides information about whether the user input meets the constraints defined for that element (e.g., `required` fields, pattern constraints, maximum length, etc.).

```
1  <input
2    id="age"
3    type="number"
4    min="18"
5    placeholder="Enter age (18+)"
6  />
7
8  <script>
9    const input = document.getElementById("age");
10
11    input.addEventListener("input", (e) => {
12      console.log(e.target.validity);
13    });
14  </script>
```

[Open Sandbox](#)

- **patternMismatch** Property: This will be `true` if the value doesn't match the specified regular expression pattern.

preventDefault() Method

- **Definition:** Every event that triggers in the DOM has some sort of default behavior. The click event on a checkbox toggles the state of that checkbox, by default. Pressing the space bar on a focused button activates the button. The `preventDefault()` method on these `Event` objects stops that behavior from happening.

```
1 <form id="form">
2   <input type="text" placeholder="Try to submit" />
3   <button type="submit">Submit</button>
4 </form>
5
6 <p id="status"></p>
7
8 <script>
9   const form = document.getElementById("form");
10  const status = document.getElementById("status");
11
12  form.addEventListener("submit", (event) => {
13    event.preventDefault();
14    status.textContent = "Form submission prevented.";
15  });
16 </script>
```

[Open Sandbox](#)

```
<form action="https://freecodecamp.org" method="GET">
  <input
    type="number"
    name="number"
    placeholder="Enter a number"
  />
  <button type="submit">Submit</button>
</form>
```

- **method Attribute:** This attribute accepts a standard HTTP method, such as GET or POST, and uses that method when making the request to the action URL. When a method is not set, the form will default to a GET request. The data in the form will be URL encoded as name=value pairs and appended to the action URL as query parameters.

```
<form action="/data" method="POST">
  <input
    type="number"
    id="input"
    placeholder="Enter a number"
    name="number"
  />
  <button type="submit">Submit</button>
</form>
```

- **enctype Attribute:** The form element accepts an enctype attribute, which represents the encoding type to use for the data. This attribute only accepts three values: application/x-www-form-urlencoded (which is the default, sending the data as a URL-encoded form body), text/plain (which sends the data in plaintext form, in name=value pairs separated by new lines), or multipart/form-data, which is specifically for handling forms with a file upload.

Assignment

- ☐ Review the Form Validation with JavaScript topics and concepts.

Please complete the assignment

Submit

Ask for Help